

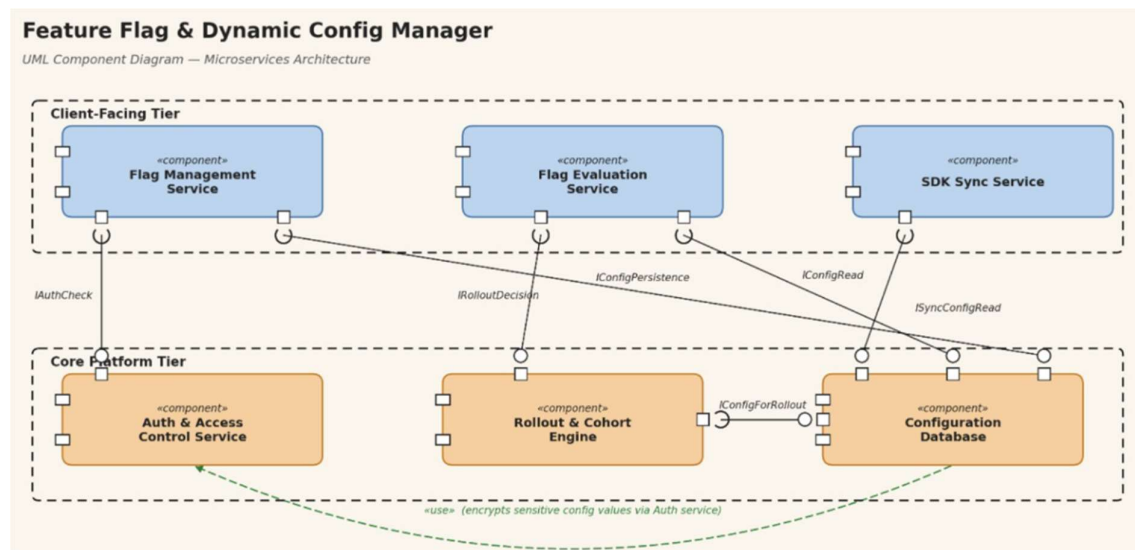
PES UNIVERSITY

SOFTWARE ENGINEERING

PES1UG24CS350

Pushkar S Kulkarni

Component Diagram



Architecture Selection

We chose Microservices Architecture for the Feature Flag & Dynamic Config Manager System.

Reason 1: Independent Scalability of Critical-Path Evaluation

NFR-001 requires the flag evaluation API to respond in under 15 ms, since it sits on client applications' critical execution paths. As a standalone service (Flag Evaluation Service), it can be scaled horizontally and optimized independently of slower-changing components such as Flag Management or the Configuration Database, without those components' load affecting evaluation latency.

Reason 2: Decoupled, Independently Deployable Workloads

The system supports distinct workloads: administrative CRUD (Flag Management Service), rollout computation (Rollout & Cohort Engine), and

real-time push updates to client SDKs (SDK Sync Service). Microservices allow each to be developed, deployed, and updated independently, so a change to flag management logic does not require redeploying or restarting the SDK sync pipeline, minimizing disruption to connected clients.

Security Advantage

Authorization is isolated into a dedicated Auth & Access Control Service, exposed through the IAuthCheck interface. Every write operation from Rollout & Cohort Engine / Configuration Database is routed through this component before it can mutate a flag or configuration override, satisfying NFR-002. Centralizing access control in one service, rather than duplicating checks across services, reduces the risk of an inconsistently enforced or bypassed authorization rule, and the modeled <<use>> dependency from the Configuration Database ensures sensitive config values are encrypted via the Auth service rather than stored in plaintext.

Performance Benefit

Because the Flag Evaluation Service and Rollout & Cohort Engine are separate, independently scalable services, evaluation traffic spikes (e.g., during a large rollout) can be absorbed by scaling only those services, rather than the entire system. This directly supports the sub-15ms latency requirement without over-provisioning components such as Flag Management or the SDK Sync Service, which experience comparatively low and infrequent traffic.

Req ID	Type	Description	Priority	Acceptance Criteria	Rationale
FR-001	Functional	The system shall evaluate user targeting rules, such as User ID hash and Beta cohort membership, and return the correct boolean feature flag state.	High	Pass: A user belonging to a configured 10% rollout cohort receives an enabled flag state, while a user outside the cohort receives a disabled flag state. Fail: A user receives a flag state inconsistent with the configured targeting rules.	Accurate feature flag evaluation is the core functionality of the system.
FR-002	Functional	The system shall allow a Software Engineer to create, update, enable,	High	Pass: The Software Engineer can create a feature flag with a unique identifier and successfully update	Feature flags must be managed

Req ID	Type	Description	Priority	Acceptance Criteria	Rationale
		disable, and delete feature flags.		its state or configuration. Fail: The requested feature flag operation is not saved or the system allows duplicate identifiers.	throughout their lifecycle.
FR-003	Functional	The system shall allow a Release Manager to configure percentage-based rollouts for a feature flag.	High	Pass: When a rollout percentage of 10% is configured, the system enables the feature only for users selected by the configured cohort evaluation mechanism. Fail: Users outside the selected rollout cohort incorrectly receive the enabled feature.	Percentage-based rollouts enable gradual and controlled feature releases.
FR-004	Functional	The system shall allow users to define environment-specific configuration overrides for development, staging, and production environments.	Medium	Pass: When different configuration values are defined for development and production, a client requesting a value for each environment receives the correct environment-specific value. Fail: A client receives a configuration value belonging to another environment.	Environment overrides allow different configurations to be safely used across deployment environments.
FR-005	Functional	The system shall synchronize updated feature flag and configuration values with connected client SDKs.	High	Pass: After a feature flag or configuration is updated, a connected client SDK receives the updated value without manual configuration changes. Fail: The SDK continues using an outdated value after synchronization.	Clients need current feature and configuration data to behave correctly.
NFR-001	Non-Functional — Performance	The feature flag evaluation API shall execute in under 15 ms under simulated peak load.	High	Pass: Performance tests show that feature flag evaluation completes in less than 15 ms under the defined simulated peak load. Fail: Evaluation takes 15 ms or longer.	Feature flag evaluation may occur on critical application paths, so overhead must remain minimal.
NFR-002	Non-Functional — Security	The system shall allow only authorized users to create, modify, enable, disable, or delete feature flags and configuration overrides.	High	Pass: Authorized users can perform permitted operations, while unauthorized users receive an access-denied response and no configuration change occurs. Fail: An unauthorized user successfully modifies a feature flag or configuration.	Unauthorized changes could affect application behavior, production stability, and software releases.